

Q1:

1. Introduction and Approach

Objective:

We aim to apply a box blur filter to grayscale images of varying sizes (1000×1000 , 2000×2000 , 3000×3000 , and 5000×5000). A box blur replaces each pixel with the average of its 3×3 neighborhood.

Method:

1. Initialization:

- Create a 2D vector (image) filled with a constant value (e.g., 128).
- Create an empty 2D vector (output) for storing blurred results.

2. Box Blur Algorithm:

- For each pixel (i, j) (excluding boundary pixels), sum up the values of the 3×3 region centered at (i, j) .
- Divide by 9 to get the average, and store the result in $\text{output}[i][j]$.

3. Parallelization with OpenMP:

- We use a `#pragma omp parallel` for directive around the main loops.
- We vary the number of threads (1, 2, 4, 8, 12, 16) and measure execution times.

4. Timing:

- We use `<chrono>` to record the start and end time of each blur operation.
- We output the elapsed time in seconds for each thread count.

2. Machine Description

- Processor: Intel® Core™ i5-8300H CPU @ 2.30GHz × 8 (example)
- Memory: 16 GB RAM
- Operating System: Ubuntu 22.04.4 LTS (64-bit)
- Compiler: g++ (with OpenMP support)

(Replace these details with your actual system info and screenshots as needed.)

3. Effect of Increasing Thread Count

As we increase the number of threads, the workload of processing each row or pixel neighborhood is split among more threads. This generally reduces the total execution time, up to a

point. Eventually, the benefit of adding more threads may be limited by factors such as:

- Overheads for thread creation and scheduling
- Memory bandwidth constraints
- Cache effects (e.g., shared caches, data locality)

In practice, we observe a significant speedup from 1 thread to multiple threads, but the gains diminish somewhat as we go beyond a certain number of threads.

4. Results

Below are the recorded execution times for each image size, showing how the performance improves as we increase the number of threads.

```

abdullah@abdullah-Latitude-5491:~/Desktop/SEMESTER_6/PDC/A03 (copy)$ g++ -fopenmp q1.cpp -o m
abdullah@abdullah-Latitude-5491:~/Desktop/SEMESTER_6/PDC/A03 (copy)$ ./m
Processing 1000x1000 image...
1 thread (serial): 0.0487775 sec
2 threads: 0.0830603 sec
4 threads: 0.0698285 sec
8 threads: 0.0453481 sec
12 threads: 0.0244887 sec
16 threads: 0.0167577 sec

-----
Processing 2000x2000 image...
1 thread (serial): 0.0648771 sec
2 threads: 0.119134 sec
4 threads: 0.120703 sec
8 threads: 0.0769301 sec
12 threads: 0.0761478 sec
16 threads: 0.117325 sec

-----
Processing 3000x3000 image...
1 thread (serial): 0.193195 sec
2 threads: 0.257566 sec
4 threads: 0.213609 sec
8 threads: 0.192019 sec
12 threads: 0.201594 sec
16 threads: 0.193119 sec

-----
Processing 5000x5000 image...
1 thread (serial): 0.564285 sec
2 threads: 0.694162 sec
4 threads: 0.548377 sec
8 threads: 0.530352 sec
12 threads: 0.501645 sec
16 threads: 0.508918 sec

-----
abdullah@abdullah-Latitude-5491:~/Desktop/SEMESTER_6/PDC/A03 (copy)$ █

```

Q3:

1. Introduction

Conway's Game of Life is a well-known cellular automaton where each cell in a 2D grid transitions between "live" and "dead" states based on the states of its neighbors. Our implementation focuses

on a 100×100 grid running for 100 generations under toroidal (wrap-around) boundary conditions. We evaluated three versions:

1. Serial Implementation
2. Parallel Implementation with Static Scheduling
3. Parallel Implementation with Guided Scheduling

This report presents the average execution times, calculated speedups, a comparison of static vs. guided scheduling, and an analysis of what happens if critical sections (or other necessary synchronization) were removed.

2. Experimental Setup

2.1 Grid Configuration

- Grid Size: 100×100
- Generations: 100
- Initialization:
 - A 10×10 block of live cells (' * ') at the center
 - All other cells dead (' . ')

2.2 Parallelization Strategy

- Double Buffering: Each new generation is computed in a separate 2D array (`newGrid`), preventing data races from in-place updates.
- OpenMP Parallel Loops: We parallelized the outer loop over the rows.
- Scheduling Modes:
 1. Static (chunk size = 1)
 2. Guided (chunk size = 1)

2.3 Execution Time Measurements

- Number of Runs: 5 runs per version
- Timer: C++ `<chrono>` high-resolution clock
- Averaging: Times from the 5 runs of each version are averaged.

(You may also include details about your CPU, memory, and OS here for completeness.)

3. Results

3.1 Average Execution Times

Below are the average times over 5 runs:

Implementat ion	Average Time (s)
Serial	0.111135

Parallel 0.0381792
(Static)

Parallel 0.0332028
(Guided)

3.2 Speedup Calculations

Speedup is calculated as:

$$\text{Speedup} = \frac{\text{Time}_{\text{Serial}}}{\text{Time}_{\text{Parallel}}}$$

1. Static Scheduling Speedup

$$\frac{0.111135}{0.0381792} \approx 2.91$$

2. Guided Scheduling Speedup

$$\frac{0.111135}{0.0332028} \approx 3.36$$

4. Comparison of Static vs. Guided Scheduling

1. Static Scheduling (chunk size = 1)

- The rows are divided into equal chunks (one row per chunk).
- This can be efficient if each row takes roughly the same time to process.

- If certain rows require slightly more computation, some threads may finish early while others remain busy, causing potential load imbalance.

2. Guided Scheduling (chunk size = 1)

- Threads request chunks dynamically, and chunk sizes can shrink as the workload is distributed.
- This often yields better load balancing, as threads that finish early can grab more chunks.
- The results show Guided scheduling is about 15% faster than Static scheduling in this particular scenario (3.36 vs. 2.91 speedup).

In this specific application (Conway's Game of Life), the workload per row is somewhat uniform. Still, even small imbalances or overhead differences can make guided scheduling slightly more efficient.

5. Impact of Removing the Critical Section

5.1 Current Synchronization Approach

- We use double buffering: the next generation is written to a separate array (`newGrid`), and then we swap or assign `newGrid` back to `grid`.
- Each thread writes to unique (`row`, `column`) positions, so no two threads ever write to the same cell simultaneously.

5.2 If Critical Sections Were Required

- In some parallel Game of Life implementations, threads might update the same array in-place or share other data structures. In such cases, critical sections or other synchronization (like barriers or atomic operations) might be necessary to avoid data races.
- Removing needed synchronization would result in:
 - Data Races: Multiple threads updating the same cell or reading partially updated neighbor states.
 - Inconsistent Final States: The final grid might differ across runs, or produce incorrect patterns that do not follow Conway's rules.

5.3 Reported Inconsistencies

- Since our approach inherently avoids race conditions (no overlapping writes), removing a hypothetical critical section does not change the outcome.
- In an alternative design (e.g., in-place updates), removing the critical section would cause inconsistent states. One might see spurious “blinks” or random changes due to cells reading updated neighbors within the same generation.

```
abdullah@abdullah-Latitude-5491:~/Desktop/SEMESTER_6/PDC/A03 (copy)$ ./m perf
===== Performance Analysis (100x100, 100 Gens) =====
Serial (avg 5 runs): 0.111135 s
Static (avg 5 runs): 0.0381792 s | Speedup: 2.91087
Guided (avg 5 runs): 0.033028 s | Speedup: 3.36486
===== END =====
abdullah@abdullah-Latitude-5491:~/Desktop/SEMESTER_6/PDC/A03 (copy)$
```